Edge cases (context limit) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `context_limit_practice.py`: Doc03 had you guess how many turns fit in a context window — this is where that guess meets a real, deliberately-triggered `BadRequestError`. Seeing the exact exception type and message once means you recognize it instantly later, instead of panicking at an unfamiliar crash. **If not:** the Build Task's "handle a too-long conversation gracefully" requirement would ask you to catch an error you'd never actually seen fire.

Solution

```
# context_limit_practice.py
from dotenv import load_dotenv
from openai import OpenAI
import openai

load_dotenv()
client = OpenAI()

# why: 200,000 repeated words reliably blows past any model's context
# window, on purpose, so the real error actually fires.
huge_input = "word " * 200_000

try:
    client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": huge_input}],
    )
except openai.BadRequestError as e:
    # how: catching the specific class documents exactly what failure this
    # code expects — a bare except Exception would also hide a real bug.
    print(f"Error type: {type(e).__name__}")
    print(f"Message: {str(e)[:200]}")
```

Expected output:

```
Error type: BadRequestError
Message: Error code: 400 - {'error': {'message': "This model's maximum
```

```
context length is 128000 tokens...
```